

SecureChain Role-Based Access Control (RBAC) & Governance Guide

This document defines the cryptographic role and permission architecture for SecureChain (SIH-26125) across smart contracts, the backend API gateway, and the web client.

1. Authoritative Admin Accounts

All previous test/sample addresses have been purged. The initial governance authority is strictly bound to the following two verified wallet addresses:

Admin #	Wallet Address	Initial Role	Permissions
1	0x8292040fb8adbe10333a74b2bf79ebfbf3b0e41c	ADMIN	Full Governance & Minting
2	0xFF00D19Db6668537116Ecd a91ac07Fa448A2223e	ADMIN	Full Governance & Minting

Note on Non-Admin Logins: Any wallet address that connects to SecureChain without prior administrative assignment automatically defaults to USER (Least Privilege principle). There is no manual role switcher in the UI—a user's role is strictly locked to their cryptographic wallet signature.

2. Role & Permission Matrix

The system defines 4 distinct tiers implemented directly in `IdentityAndAccessManager.sol`:

Action / Capability	ADMIN	MANAGER	AUDITOR	USER
Role Assignment & Revocation	✓	—	—	—
Identity Registration & Status Update	✓	—	—	—
ERC-721 Asset Minting	✓	—	—	—
Initial Asset Allocation	✓	—	—	—
Policy-Controlled Asset Transfer	✓	✓	—	Policy (Owned only)
Confidential Document Upload	✓	✓	—	✓
Document Versioning & Revisions	✓	✓	—	✓
Cryptographic 4-Step Verification	✓	✓	✓	✓
Audit Trail Inspection	✓	✓	✓ (Full)	Read (Self only)
Social Recovery Configuration	✓	—	—	—

3. How Users Receive Roles

1. User Connects Wallet: A user navigates to SecureChain and clicks Connect Wallet (MetaMask, Coinbase, Trust Wallet).

2. Deterministic Lookup: The application reads `getRoleForWallet(address)`:

- If the wallet matches one of the designated Admin addresses, they are authenticated as ADMIN.
- If an Admin has previously assigned them MANAGER or AUDITOR, they receive that role.
- If unassigned, they are safely assigned USER.

3. Restricted View: The sidebar and page router dynamically lock routes. Non-Admins cannot access `/rbac`, `/identity`, or `/recovery`.

4. How an Admin Assigns a Role to a Wallet

To grant elevated privileges (e.g. appointing a new Officer as MANAGER or an inspector as AUDITOR):

1. Connect with an authorized ADMIN wallet (e.g. 0x8292040fb8adbe10333a74b2bf79ebfbf3b0e41c or 0xFF00D19Db6668537116Ecda91ac07Fa448A2223e).

2. Navigate to Access Control (RBAC) (/rbac) from the sidebar.

3. Locate Section 2: Assign Role by Wallet Address:

- Wallet Address: Paste the target Ethereum address (0x . .).
- Role: Select ADMIN, MANAGER, or AUDITOR.
- Click Assign Role.

4. On-Chain & Registry Execution:

- The role is saved immediately to the local governance registry.
- If connected to the Polygon gateway, an on-chain transaction calls:

```
```solidity
```

```
IdentityAndAccessManager.grantRole(bytes32 role, address acct);
```

```
```
```

- As soon as that wallet connects, it will instantly possess the new privileges.
-

5. How an Admin Revokes / Removes a Role

To downgrade an address or strip elevated privileges:

1. Connect with an ADMIN wallet and open Access Control (RBAC) (/rbac).

2. Scroll to Section 3: Assigned Wallet Role Directory.

3. Find the wallet row and perform either of the following actions:

- Option A (One-Click Revoke): Click the red Revoke button (UserX icon). The wallet is immediately removed from the privileged directory and downgraded to USER.
- Option B (Dropdown Change): Select USER from the role dropdown.

4. On-Chain Execution:

- The registry removes the mapping.
- If connected to the Polygon gateway, it executes:

```
```solidity
```

```
IdentityAndAccessManager.revokeRole(bytes32 role, address acct);
```

```
```
```

6. Smart Contract Invariants

1. Non-Escalation: A non-admin cannot call grantRole or revokeRole. The contract checks:

```
```solidity
```

```
modifier onlyAdmin() {
```

```
require(_roles[ADMIN_ROLE][msg.sender], "IAM: not admin");
```

```
_;
```

```
}
```

```
```
```

2. Zero-Admin Guard: The contract cannot be initialized with a zero address.
3. Auditor Immutability: The AUDITOR role has zero state-mutating permissions. An auditor can never mint tokens, alter documents, or transfer assets.